Distilling and Reinforcing Macedonian Poetic Style Transfer: Supervised Fine-Tuning and GRPO on a 4B Student

TODO: author name

October 18, 2026

Abstract

A large instruction-tuned model (Gemma 4 31B), given a heavily scaffolded prompt (an author style profile in prose, a vocabulary palette, exemplar lines and explicit structural targets), can rewrite a Macedonian poem in a named poet's style. This report covers the attempt to obtain the same behaviour from a small, locally runnable model that is given only the source poem and the target author's name. The teacher's outputs on 1,146 train-split pairs are distilled into Qwen3-4B by LoRA supervised fine-tuning (SFT); the distilled model is then trained further with Group Relative Policy Optimization (GRPO) against a reward built from the project's own stylometric profiles, a line-level content-similarity score and multiplicative gates against copying, repetition and malformed output. SFT teaches the task format completely but collapses ten target voices into one averaged voice dominated by the corpus's two largest authors. GRPO (v1) raises the reward from 0.254 to 0.406, removes the repetition SFT showed under plain sampling and doubles content preservation, but it halves author identification (on the reward classifier and, less sharply, on a held-out judge) and learns to copy single source lines at the edge of the copy gate. The Gemma teacher, scored on the same pairs, is still identified as the requested author three to eight times as often as either student. GRPO-v2 changes only the reward to address both findings, and a second student of matched size from another family (Gemma 4 E4B) separates properties of Qwen from properties of the method. GRPO-v2 keeps author identification close to SFT's on both classifiers instead of losing half of it, but only by holding the policy on the two largest authors, not by learning the other eight, and it gives back most of v1's repetition fix; for eight of the ten targets neither student is ever identified as the requested author, which leaves GRPO's group-relative update nothing to amplify. **[pending: Gemma result, one sentence]**

Contents

1 Overview **2**

2 Data **3**
2.1 Splits . . . 3
2.2 Target authors . . . 3
2.3 Synthetic training data . . . 3

3 Supervised fine-tuning **4**
3.1 Task format . . . 4
3.2 Training . . . 4
3.3 Training run . . . 5
3.4 Runtime environment . . . 5

4 Evaluation methodology **5**
4.1 Protocol . . . 5
4.2 Metrics . . . 6
4.3 Copying has to be measured . . . 6
4.4 Two author classifiers . . . 6

5 SFT results **7**
5.1 Against the base model . . . 7
5.2 Style collapse . . . 7
5.3 Degenerate decoding versus a degenerate policy . . . 8
5.4 Defects found during the evaluation . . . 9
5.5 What SFT leaves for GRPO . . . 9

6 GRPO: method and reward **10**
6.1 Why the SFT adapter is merged first . . . 10
6.2 Prompts . . . 10
6.3 Style reward . . . 10
6.4 Content reward . . . 11
6.5 Gates . . . 11
6.6 Choosing the weights from within-group spread . . . 11
6.7 Validating the reward before training . . . 12
6.8 Training configuration . . . 12

7 GRPO v1 results **13**
7.1 Target-style match went down . . . 14
7.2 Partial copying . . . 15
7.3 Training dynamics . . . 15
7.4 The teacher on the same pairs . . . 15

8 GRPO v2: correcting the reward **16**
8.1 Results . . . 17

9 A second student: Gemma 4 E4B **18**
9.1 Purpose . . . 18
9.2 Implementation . . . 18
9.3 Results . . . 20

10 Limitations **20**

11 Reproducibility **21**

1 Overview

The project's corpus is 2,711 Macedonian poems and song lyrics (Cyrillic). Two earlier style-transfer approaches precede this work: a token-substitution pipeline that splices author vocabulary into masked slots, and a generative pipeline that sends Gemma 4 31B one prompt combining an English summary of the source poem, its content keywords, the target author's style profile in prose, a relevance-filtered vocabulary palette,

tone-exemplar lines and structural targets (lines, stanzas, tokens per line). The second produces fluent, recognisably author-specific poems, but it depends on a 31B model behind a rate-limited API (about three minutes per poem including the model's obligatory reasoning) and on the whole stylometric pipeline at inference time.

The question this report addresses is how close a 4B model can get to that teacher when it is given only

$$\textit{source poem} + \textit{target author's name} \;\rightarrow\; \textit{new poem}.$$

The work is three stages, each consuming the previous one's output:

1. **Synthetic data** (§2). The teacher pipeline, unmodified, writes transfers for (source, target) pairs drawn from the train split only.
2. **SFT** (§3, §5). LoRA fine-tuning of Qwen3-4B on those pairs, in a compact chat format that contains none of the teacher's scaffolding.
3. **GRPO** (§6–§8). Reward-driven fine-tuning of the SFT model against a reward computed only from the generated poem, the source poem and the target author.

§9 repeats stages 2 and 3 with a second student (Gemma 4 E4B) to test whether the failures found with Qwen belong to the student or to the method.

2 Data

2.1 Splits

`make_splits.py` assigns every poem to `train`, `val` or `test`, 80/10/10 per author, seeded (seed 42). The unit is the poem, not the (author, title) pair, which is not unique in this corpus (Блаже Конески alone has four distinct poems titled `ПЕСНА`). Authors with fewer than three poems cannot be split three ways and go entirely to `train`; every other author has at least one poem in each split. Over the 1,239 poems of the 38 profiled authors this gives 987 train, 126 val and 126 test poems. Every later stage reads the split from this one file; nothing recomputes it.

2.2 Target authors

The ten target authors are the ten authors with the most poems among those with a style profile (ties broken alphabetically): Блаже Конески, Пијан Славеј, Ефтим Клетников, Ристо Лазаров, Веле Смилевски, Братислав Ташковски, Влада Урошевиќ, Адем Гајтани, Славко Јаневски and Катица Ќулавкова. They were chosen for having enough real poems to profile reliably, not by a diversity criterion; distinct voices follow from their being different poets.

2.3 Synthetic training data

`generate_synthetic.py` calls the teacher pipeline (`llm_style_transfer.run_transfer`) in a loop over sampled (source, target) pairs from the train split, with one sample per pair. The same set of source poems is reused across all target authors, so that the training data contrasts authors on identical content rather than on different poems. The run produced 1,146 examples: 130 distinct source poems, 108–120 examples per target author, with self-transfer pairs skipped and failed pairs (API errors, empty answers) logged separately rather than retried into the data.

Generation is the throughput bottleneck: the Gemini API serves a few requests per minute, and Gemma 4 always reasons before answering, with reasoning length varying between about 2k and 16k tokens for the

same prompt. The batch was therefore run as one process per target author in parallel, each writing its own part file, merged afterwards on the key (source poem, target author, sample index). The output file is also the resume record, so an interrupted run never regenerates a finished pair.

The same procedure on the val split (50 val poems $\times$ 10 targets, self pairs skipped, 468 pairs) gives the teacher's output on exactly the pairs the students are evaluated on (`data/synthetic/teacher_val.csv`). It is evaluation data only and is never trained on. All 468 pairs are complete.

3 Supervised fine-tuning

3.1 Task format

The student is trained on a deliberately minimal chat:

- *system*: "You rewrite Macedonian poems in a different poet's style. Given a source poem and a target author, write a new poem in Macedonian, in Cyrillic script, that conveys the source poem's content and imagery in the target author's voice."
- *user*: `Source poem:\n{poem}\n\nTarget author: {name}`
- *assistant*: the teacher's poem.

None of the teacher's scaffolding — the summary, keywords, style profile, palette, exemplars or structural targets — is in the student's prompt. Whatever the student knows about an author it has to learn from the 108–120 teacher poems written for that author. The same module (`sft_data.py`) renders the prompt for training, for validation generation and for GRPO, through the tokenizer's own chat template, so the three cannot drift. Qwen3 is used as a non-reasoning model (`enable_thinking=False`).

3.2 Training

LoRA on `Qwen/Qwen3-4B` with TRL's `SFTTrainer` and PEFT:

LoRA rank / α / dropout	16 / 32 / 0.05
Target modules	`q,k,v,o,gate,up,down_proj` (all attention and MLP projections)
Epochs	3
Batch	4 per device $\times$ 4 accumulation $= 16$
Learning rate	2×10^{-4}, cosine, 3 % warm-up
Max sequence length	1024 tokens
Precision	bf16, gradient checkpointing
Loss	assistant turn only (`assistant_only_loss`)
Seed	42

The target modules were checked against the transformers Qwen3 source rather than taken from a generic Llama default; Qwen3's `q_norm` and `k_norm` are RMSNorm layers and are not adapted. The loss is masked to the assistant turn: TRL patches generation markers into Qwen3's chat template for this. With that template the masked span also contains Qwen3's constant empty reasoning block (`<think>\n\n</think>`) and the newline after the end-of-turn token, which are trivially learnable and do not affect the poem.

3.3 Training run

The run took 210 optimizer steps (70 per epoch) and processed 2.17M tokens, prompts included (6.1×10^{16} FLOPs). The loss, over assistant tokens only, from the run's `trainer_state.json`:

Step	Epoch	Loss	Token accuracy	Entropy
10	0.14	2.871	0.454	1.93
40	0.57	1.901	0.550	1.90
70	1.00	1.717	0.583	1.70
80	1.14	1.572	0.605	1.60
140	2.00	1.496	0.626	1.50
150	2.14	1.367	0.654	1.41
210	3.00	1.341	0.658	1.36

Each row is the mean over the preceding 10 steps. Token accuracy is the share of assistant tokens predicted exactly; entropy is the mean entropy of the predicted next-token distribution.

Most of the loss reduction happens in the first epoch (2.87 → 1.72). After that, the loss falls in steps at the epoch boundaries and stays almost flat within an epoch: it drops 0.15 from step 70 to step 80 and 0.13 from step 140 to step 150, but only 0.03 over the 60 steps of the third epoch after step 150. Steps at epoch boundaries are the usual sign that the model recognises examples it has already seen. With 1,146 examples and three passes, part of the second and third epochs' gain is memorisation of the training poems rather than better generalisation. The run has no held-out loss to tell the two apart: the evaluation split was configured, but the synthetic dataset contains only train rows, so no evaluation step ran. The teacher's val-pair outputs (`teacher_val.csv`, §7.4) were generated later and could now serve as that held-out set. The final loss of 1.34 corresponds to a per-token perplexity of about 3.8 on the teacher's poems.

The run took 881 s (about 15 minutes) on one NVIDIA A100-SXM4-80GB.

3.4 Runtime environment

Training and generation run on a single A100 in a Slurm cluster, inside a Singularity image built from the project's own `pyproject.toml` and `uv.lock` (`uv sync --frozen` at build time). The image is therefore the same environment as local `uv run`, with no separate CUDA base image: the PyPI `torch` wheel bundles its CUDA runtime, and only the driver comes from the host (`--nv`). One bind mount maps the repository into the container, so a job reads and writes the same `data/` and `models/` as local runs.

4 Evaluation methodology

4.1 Protocol

Every model is evaluated on the same 468 val pairs (50 val poems × 10 targets, self pairs skipped), with the same prompt, the same per-item seeds and plain sampling unless stated otherwise ($T = 0.7$, top-$p = 0.9$, top-k 20, at most 512 new tokens). Models compared with each other are generated back to back in one job on one GPU: sampling draws from the device's own RNG stream, so comparing a CPU run with a GPU run would confound the model difference with a change of random generator. Results are reported over all rows and over the *paired* view (only pairs present in every compared run).

4.2 Metrics

All metrics are computed by one module, `style_metrics.py`, and are oriented higher-is-better and mostly bounded in $[0, 1]$, because the GRPO reward uses the same functions; a reward that disagreed with the evaluation metric would be a defect.

Group	Metrics
Degeneracy	distinct-1/2/3; line uniqueness; maximum repeats of one line. An output is *degenerate* if distinct-1 < 0.40 or line uniqueness < 0.80.
Format	Cyrillic ratio; no markdown or commentary.
Structure	fit of line count, stanza count and tokens per line to the target author's profile.
Target style	`classifier.pkl` probability of and top-1 for the target author; target TF-IDF vocabulary hit rate; the held-out judge (§4.4).
Content	content-word recall from the source; line novelty and copy novelty (§4.3).

Two reference rows anchor the numbers: the teacher's output, and the target authors' real poems (`--corpus-reference`), which show what each metric scores on human text.

4.3 Copying has to be measured

The untuned base model's dominant response is not a failed style transfer. It reproduces the source poem verbatim under a markdown heading, followed by a paragraph explaining the target author's style. That output has near-perfect content preservation and, because it is real human poetry, excellent diversity and fluency. Under any combination of diversity, fluency and content metrics, "change nothing" would score highest. `line_novelty` (share of generated lines that are not verbatim source lines after normalising punctuation and case) and its fuzzier successor `copy_novelty` (§6.5) are the guard.

4.4 Two author classifiers

`classifier.pkl` is the project's character-(2,4)-gram author classifier. It was fitted on the whole corpus, so on the target authors' real poems it scores top-1 1.000 by memorisation; the same recipe's held-out accuracy under 5-fold cross-validation, 0.883, is the honest ceiling. GRPO also uses this classifier's target probability as a reward component, so after GRPO its score is no longer independent evidence: a policy trained to raise it will raise it whether or not the poem reads as the author.

The *judge* (`judge_classifier.py`) is a second classifier used only for evaluation and differs from the reward classifier in data, features and model: it is fitted on train-split poems only (authors with at least five train poems, 23 authors, 963 poems), on word unigrams and bigrams, with multinomial logistic regression. Its top-1 on the 172 val and test poems of the ten target authors is 0.663, which is the ceiling for real unseen poems; in the real-poem row the judge's columns are blank for train poems, which it has seen. A test forbids importing the judge from any reward module. A gain in classifier top-1 without a gain in judge top-1 would mean the reward classifier was gamed.

5 SFT results

5.1 Against the base model

Metric (higher is better)	Base Qwen3-4B	LoRA-SFT (tuned)	Gemma teacher*	Real poems
Task compliance				
line novelty (not copied from source)	0.224	**1.000**	1.000	—
no markdown / commentary	0.592	**1.000**	1.000	0.960
structural fit to target	0.680	**0.848**	0.950	0.768
Target-style match				
classifier top-1 = target author	0.009	**0.141**	0.589	0.883†
classifier probability of target	0.024	**0.115**	0.381	—
target TF-IDF vocabulary hit rate	0.005	**0.044**	0.311‡	0.050
Content preservation				
content-word recall from source	0.901§	0.173	0.176	—
Degeneracy				
distinct-1	0.635§	0.515	0.768	0.770
line uniqueness	0.895	0.904	0.999	0.969
degenerate outputs (% of rows)	18.6%	25.0%	0.2%	6.6%

* the teacher's 1,146 *train* rows (its training data); the teacher on the val pairs is in §7.4. † held-out 5-fold CV, not the memorising shipped model. ‡ the teacher was *handed* this palette in its prompt, so it exceeds human text; the metric measures palette uptake, not quality. § inflated by verbatim copying.

Fine-tuning worked on every axis the task is defined on. The tuned model writes original lines instead of copying (1.000 vs. 0.224), never emits markdown or commentary (1.000 vs. 0.592), fits the target author's structure better (0.848 vs. 0.680), and is 15 times more likely to be identified as the requested author (0.141 vs. 0.009). Its content-word recall, 0.173, matches the teacher's 0.176: the teacher's degree of content preservation was distilled faithfully, low as it is.

The base model's diversity advantage is an artifact. 87.8 % of its outputs are mostly copied from the source, and the copied human poetry carries the diversity. On the 12.2 % of rows where it composes something new, its distinct-1 is 0.532 — indistinguishable from the tuned model's 0.515 — with a classifier top-1 of 0.035. Fine-tuning did not cost diversity; it removed the copying that disguised the base model's own lack of it.

5.2 Style collapse

Broken out per target author, the tuned model does not have ten voices:

Target author	Base	LoRA-SFT	Gemma teacher
Блаже Конески	0.000	0.574	0.899
Пијан Славеј	0.000	0.395	0.565
Ефтим Клетников	0.021	0.170	0.655
Ристо Лазаров	0.000	0.106	0.608
Веле Смилевски	0.000	0.104	0.881
Братислав Ташковски	0.000	0.022	0.579
Влада Урошевиќ	0.000	0.021	0.816
Адем Гајтани	0.000	0.021	0.345
Славко Јаневски	0.000	0.020	0.183
Катица Ќулавкова	0.065	0.000	0.395

Classifier top-1, per requested target author.

The tuned model's aggregate is carried by Блаже Конески and Пијан Славеј, the corpus's two largest authors (182 and 160 poems); asked for Катица Ќулавкова it succeeds zero times in 46. Whatever author is requested, the classifier reads 35.0 % of its output as Конески. The teacher is differentiated across the range (0.183–0.899). The SFT data is balanced across targets (108–120 examples each), so the imbalance does not come from the number of training examples per target.

Pooled vocabulary at a matched 30,000-token budget confirms the collapse is corpus-level, not only per-poem repetition: 3,087 distinct word types for the tuned model, against 5,376 for the teacher and 9,060 for real poems by the same authors. It is syntactic as well as lexical: the tuned model falls into a fixed line template, ⟨noun⟩ ⟨adverb⟩ ⟨verb⟩ *сега*, visible even in its best-scoring output (classifier probability 0.785 for Пијан Славеј, a poem in which *Мераци брзо се вртат сега* occurs four times).

5.3 Degenerate decoding versus a degenerate policy

The repetition and the collapse could be one fault or two. A decoding ablation separates them: the same adapter, 56 val pairs, the same seeds, four decoding settings.

Metric	Plain sampling	Repetition penalty 1.1	Repetition penalty 1.2	No repeated 4-gram
distinct-1	0.503	0.807	0.895	0.751
line uniqueness	0.876	0.998	1.000	1.000
degenerate outputs	30.4%	**0.0%**	**0.0%**	**0.0%**
classifier top-1 = target	0.125	0.107	0.161	0.125
paired change vs. plain	—	−0.018	+0.036	+0.000
content-word recall	0.125	0.078	0.031	0.088
structural fit	0.859	0.837	0.813	0.837

The repetition is a decoding artifact. A repetition penalty of 1.1 removes degenerate output entirely and lifts distinct-1 from 0.503 to 0.807, past both the teacher and the real poems. Banning repeated 4-grams does the same at 0.751, closest to human text.

The style collapse is not. None of the settings moves target-style match: paired changes in classifier top-1 are noise around zero. That deficit is in the policy, and only training can move it.

Cost. A repetition penalty cannot tell a looped word from a word legitimately carried over from the source, so content recall falls (0.125 → 0.078 at 1.1, → 0.031 at 1.2). The 4-gram ban constrains sequences and costs less (0.088), at the price of forbidding deliberate refrains, which real poems use (line uniqueness 0.969).

Full-scale confirmation. The repetition-penalty-1.1 setting was rerun on all 468 val pairs for both the base and the SFT model (`data/validation_generations/reppen-1.1/`):

	Base Qwen3-4B		LoRA-SFT	
Metric	plain	penalty 1.1	plain	penalty 1.1
distinct-1	0.635	0.675	0.515	**0.799**
line uniqueness	0.895	0.928	0.904	**0.999**
max. repeats of one line	1.80	1.72	2.04	**1.03**
classifier top-1 = target	0.009	0.024	0.141	**0.182**
classifier probability	0.024	0.028	0.115	**0.127**
judge top-1 = target	0.064	—	0.158	**0.175**
target TF-IDF hit rate	0.005	0.004	0.044	**0.079**
structural fit	0.680	0.670	**0.848**	0.834
content-word recall	0.901[§]	0.829[§]	**0.173**	0.114
line novelty	0.224	0.361	1.000	1.000

The 56-pair result holds at full scale. The penalty removes the SFT model's repetition (line uniqueness 0.999, at most 1.03 repeats of a line) at a cost of a third of its content recall. On 468 pairs it also raises classifier top-1 from 0.141 to 0.182, a larger move than the noise-level changes of the 56-pair ablation, and the held-out judge moves the same way (0.158 → 0.175); the TF-IDF hit rate almost doubles as well, consistent with the penalty pushing the model off its few looped words and onto a wider vocabulary that includes more of the target's. Under the penalty the base model still copies most of its source (line novelty 0.361) and is no better at the task. All GRPO comparisons below use plain sampling, so that the policy, not the decoder, is what is being measured.

5.4 Defects found during the evaluation

Validation generation ran on the CPU. `from_pretrained` places the model on the CPU and `generate()` then runs there silently. The first validation run took 131 s per generation (17 hours for 468 rows) on a node with an idle A100, against 15.5 s once the model is moved to the device. The loader now logs the device the model is on.

6.2 % of training examples were truncated. Measured with the Qwen3 tokenizer, the 1,146 training examples have median length 622 tokens, 99th percentile 2,364 and maximum 4,031; at `max_seq_length` 1024, 71 examples (6.2 %) lose the end of their assistant turn and with it the end-of-turn token, so they train the model to continue past where a poem should stop. This is a plausible contributor to the looping. 2048 would cover about 99 % of examples. Qwen's SFT was *not* retrained at 2048, because that would change the start policy of both GRPO runs; it is a stated limitation (§10), and the Gemma student is trained at 4096 tokens, which truncates nothing (§9).

5.5 What SFT leaves for GRPO

Repetition needs a decoding control or a policy change, not more SFT. Style collapse is what the GRPO stage exists to address: SFT taught the task format but not the separation between authors. The style reward

therefore has to be computed per target author, so that a policy reverting to Конески is not paid for it, and content has to be a reward term in its own right, because imitating a teacher that preserves little content cannot raise it.

6 GRPO: method and reward

GRPO (TRL's `GRPOTrainer`) samples G completions per prompt, scores each with the reward, and normalises rewards within the group, so the update pushes probability towards the above-average members of each group. There is no value network; the group mean is the baseline. The reward sees three inputs only: the generated poem, the source poem and the target author. The teacher's output is never a reward target, and no val or test poem enters the prompts, the reward calibration or any in-training evaluation.

6.1 Why the SFT adapter is merged first

With a LoRA policy, TRL obtains the KL reference by switching the trainable adapter off. Continuing to train the SFT adapter would therefore anchor the KL term to the *untuned* base model, the one that copies its source under a markdown heading. Instead the SFT adapter is merged into the base weights (`merge_sft_adapter.py`, which records the adapter's hash) and a fresh LoRA is trained on top. A fresh LoRA starts as the identity ($B = 0$), so the start policy and the KL reference are both exactly the SFT model. The merged model reproduces the unmerged adapter's validation scores to within sampling noise ($R = 0.254$ for both).

6.2 Prompts

Train-split poems $\times$ the ten targets, self-transfer skipped, seeded shuffle, rendered with the SFT prompt format. Prompts longer than 1,024 tokens are dropped, never truncated.

6.3 Style reward

The style reward reuses the definitions of the per-author style profiles, computed per poem. The per-poem code was extracted into `style_features.py`, which the profiler now imports; the refactored profiler's output is byte-identical to the original's. Generated poems are tagged with `tagging.tag_poem`, the corpus build's tagger with its hand corrections and filtering.

Every feature is scored against the target author's *per-poem* distribution over their train poems (`reward_calibration.json`), because a single poem varies a lot around its author's mean. For a scalar feature x_f,

$$s_f = \exp\!\left(-\tfrac{1}{2} z_f^2\right), \qquad z_f = \frac{x_f - \mu_{a,f}}{\max(\sigma_{a,f},\, 0.5\,\sigma_{\text{corpus},f})}, \tag{1}$$

where the floor stops authors with few or unusually uniform poems from zeroing every small miss. For the POS-bigram and rhyme-ending distributions, the cosine c to the author's aggregate distribution is scored one-sidedly against the author's own leave-one-out self-similarity (μ_c, σ_c): $s = 1$ for $c \geq \mu_c$, otherwise $\exp(-\frac{1}{2}((c-\mu_c)/\sigma_c)^2)$. The classifier's target probability is used as it is, and TF-IDF palette hits are divided by the author's 90th-percentile hit rate on real poems (the raw rate is about 0.05 even for human text). Scores are averaged within groups and R_{style} is the weighted mean of the groups:

Group	Features	v1 weight	v2 weight
POS profile	`pct_NOUN/VERB/ADJ/ADV/PROPN`	1.0	1.0
lexical diversity	MATTR over lemmas (TTR below the 200 window)	1.0	1.0
structure	tokens/line, lines, stanzas, lines/stanza	1.0	1.0
POS bigrams	bigram distribution	1.0	1.0
classifier	`clf_target_prob`	1.0	**5.6**
rhyme	rhyme-ending distribution	0.5	0.5
TF-IDF	palette hits / p90	0.5	0.5

6.4 Content reward

Content is a line-level, BERTScore-style F1 against the source poem over `paraphrase-multilingual-mpnet-base-v2` embeddings: recall averages each source line's best cosine match among the generated lines, precision the reverse. Working per line avoids the model's 128-token limit and stops a poem that covers only the first stanza from earning full recall. Because every line takes its best match, unrelated poems already score high: the mean F1 over 2,000 random cross-author train pairs is $b = 0.553$. The reward is rescaled so that this level is zero, $R_{\text{content}} = \text{clip}\big((F_1 - b)/(1 - b), 0, 1\big)$.

6.5 Gates

Three behaviours are cheaper for the policy than doing the task. Each is gated multiplicatively rather than added as another objective, so it cannot be traded against the rest of the reward:

- *Copying.* R_{content} is multiplied by `copy_novelty`, the share of generated lines that are not near-copies. A line of at least three tokens is a copy if at least 75 % of its word types occur in one source line. Containment is used rather than Jaccard because on this corpus's $\sim$5-token lines one swapped word gives Jaccard $4/6 = 0.67$, which a 0.7 threshold would let through. v2 raises the gate to a power, $\texttt{copy_novelty}^k$, $k = 3$ (§8).
- *Repetition.* R_{style} is multiplied by line uniqueness (0.97 on real poems).
- *Format.* Both terms are multiplied by validity $=$ `is_cyrillic` $\cdot$ `no_chat_markup` $\cdot$ `is_nonempty`. The markup test is narrower than the reported `no_markup`: bullet and numbered-line patterns fire on 49 of the 1,239 real poems (dialogue lines opening with "- "), and as a hard gate zeroed a real target poem in the fixtures. Bold, heading and code-fence markup alone catch 186 of the untuned model's 191 markup outputs and occur in one real poem.

The total is $R = \alpha\, R^g_{\text{style}} + \beta\, R^g_{\text{content}}$, where g marks the gated terms.

6.6 Choosing the weights from within-group spread

Under group-normalised advantages, which term drives the update depends on its spread *within* a group, not on its mean. `evaluate_reward.py --groups` samples $G = 4$ SFT completions per train prompt exactly as training does and reports each component's within-group standard deviation (`data/grpo/reward_groups.csv`). The gated style term's is 0.113 and the gated content term's only 0.042, so at equal weights style would dominate the advantage almost three to one. The training runs therefore use $\alpha = 0.3$, $\beta = 0.7$. Within style, the classifier's spread is 0.051, against 0.26 and 0.31 for lexical diversity and POS bigrams; this measurement is what v2's classifier weight is derived from.

6.7 Validating the reward before training

`evaluate_reward.py --fixtures` scores seven hand-built candidates for four train (source, target) pairs and checks orderings that any usable reward must satisfy. All hold:

Candidate	R_{style}	R_{content}	R^{g}_{content}	R
real poem by the target	0.75–0.90	0.00–0.05	0.00–0.05	0.38–0.45
verbatim source	0.49–0.63	1.00	0.00	0.25–0.32
near-copy (one word per line)	0.43–0.58	0.50–0.72	0.00	0.22–0.29
Gemma teacher output	0.61–0.83	0.00–0.26	0.00–0.26	0.35–0.51
one line repeated 12×	0.19–0.29	0.00–0.37	0.00	0.01
markdown answer in English	0.17–0.36	0.00	0.00	0.00
empty	0.08–0.11	0.00	0.00	0.00

R at $\alpha = \beta = 0.5$.

The target's own poem gets the highest style score, and both copies lose their whole content reward to the gate. The table also shows the reward's main weakness: the teacher's raw content score is low (0.00–0.26), so after the baseline rescaling a genuine paraphrase barely clears an unrelated poem. On the 468 val pairs the reward ranks SFT above the untuned base even though the base's raw content score is far higher, because the gates remove it:

Metric	untuned base	SFT
copy novelty	0.197	0.993
R_{style} / gated	0.487 / 0.272	0.425 / 0.387
R_{content} / gated	0.817 / 0.074	0.121 / 0.120
R	0.173	0.254
rows with $R = 0$	39.7 %	0.0 %

6.8 Training configuration

Start policy / KL reference	SFT merged into Qwen3-4B
LoRA rank / α / dropout	16 / 32 / 0
Group size G	4
Batch	8 completions × 4 accumulation (8 prompts per step)
Learning rate	10^{-5} constant, 10 warm-up steps
Steps	500 (16,000 scored completions)
Sampling temperature	0.9
Max completion	512 tokens, truncated completions masked from the loss
KL coefficient β_{KL}	0.04
Reward weights	$\alpha = 0.3$, $\beta = 0.7$
Seed / precision	42 / bf16

A smoke run (3 steps, 4 prompts, learning rate 10^{-4}) precedes every long run and asserts finite loss and rewards, non-constant rewards within a group, LoRA B moved off zero, and a changed log-probability of a fixed completion with the adapter on versus off. The completion cap matters here: SFT completions have a median of 291 tokens, and with truncation masking a short cap would zero the loss of every completion. Every run writes its arguments, git commit, merge provenance, calibration hash

and library versions to `run_config.json`, and every reward component of every scored sample to `reward_components.jsonl`.

On one NVIDIA A100-SXM4-80GB, the 500 steps took 28,540 s (7.9 h) for v1 and 29,920 s (8.3 h) for v2, about a minute per step: generating the 32 completions per step and scoring each with the reward (which POS-tags every poem with CLASSLA on the CPU) dominate, not the gradient update.

7 GRPO v1 results

Evaluated on the 468 val pairs against its start policy, generated back to back on one GPU with the same seeds and plain sampling. All R columns are recomputed at the neutral $\alpha = \beta = 0.5$, not the training weights, so they do not favour the trained policy's own objective.

Metric (higher is better unless noted)	Base Qwen3-4B	SFT (merged)	GRPO v1	Gemma teacher*	Real poems
Reward ($\alpha = \beta = 0.5$)					
R^g_{style}	0.272	0.384	0.511	**0.686**	—
R^g_{content}	0.074	0.123	**0.302**	0.105	—
R	0.173	0.254	**0.406**	0.395	—
Degeneracy					
distinct-2	0.863	0.785	0.896	**0.970**	0.948
distinct-3	0.898	0.885	0.959	**0.995**	0.971
line uniqueness	0.895	0.900	0.971	**0.998**	0.969
max. repeats of one line (lower)	1.80	2.05	1.37	**1.03**	1.34
degenerate outputs (lower)	18.6%	30.1%	5.3%	**0.2%**	6.6%
Content preservation					
content-word recall from source	0.901§	0.174	**0.338**	0.159	—
copy novelty	0.197	0.995	0.956	**0.999**	—
Target-style match					
classifier probability of target	0.024	0.120	0.083	**0.404**	—
classifier top-1 = target author	0.009	0.158	0.079	**0.641**	0.883†
judge probability of target	0.061	0.098	0.085	**0.232**	0.298¶
judge top-1 = target author	0.064	0.158	0.130	**0.506**	0.663¶
target TF-IDF vocabulary hit rate	0.005	0.043	0.043	0.302‡	0.050
structural fit	0.680	0.846	0.831	**0.950**	0.768

* teacher on the same 468 val pairs (§7.4). § inflated by verbatim copying. † held-out 5-fold CV. ‡ the teacher is handed the palette. ¶ val and test poems only, which the judge has not seen. Bold marks the best model output per row.

The reward rose, and it was not gamed on the axes it gates. GRPO scores higher than its start policy on 94.4 % of the paired items. The content term more than doubled (0.123 to 0.302), and no output has a zero gated content reward, against 18.4 % for SFT. The degeneracy that §5.3 attributed to decoding is largely gone *under plain sampling*: 5.3 % degenerate outputs against 30.1 %, and line uniqueness (0.971) and repetition (1.37) now match real poems. This came from the policy, not a decoding control, and — unlike the repetition penalty, which cut content recall — together with higher content recall. Format compliance is unchanged (100 % Cyrillic, no markup).

Where the style gain came from. Between the first and last 50 training steps (`reward_components.jsonl`), lexical diversity rose 0.49 → 0.71, POS bigrams 0.64 → 0.84 and POS profile 0.60 → 0.67; structure, rhyme and TF-IDF stayed flat, and the classifier probability *fell*, 0.102 → 0.078. These are exactly the groups with the largest within-group spread before training. A component that barely varies among a prompt's four samples barely steers a group-normalised update: at equal group weight the classifier contributed a fifth to a sixth of the within-group variation either of those groups did, and all of style entered R at only $\alpha = 0.3$.

7.1 Target-style match went down

GRPO is identified as the requested author less often than its start policy (top-1 0.158 → 0.079), and the classifier probability is lower for 9 of the 10 targets:

Target author	SFT	GRPO v1
Блаже Конески	0.745	0.404
Пијан Славеј	0.372	0.186
Ефтим Клетников	0.149	0.021
Веле Смилевски	0.125	0.062
Адем Гајтани	0.085	0.000
Ристо Лазаров	0.064	0.043
Катица Ќулавкова	0.022	0.022
Братислав Ташковски	0.022	0.022
Влада Урошевиќ	0.021	0.021
Славко Јаневски	0.000	0.020

Most of the aggregate loss is on the two authors that carried SFT's score, and the share of all outputs the classifier assigns to Конески falls from 36.3 % to 21.8 %. GRPO has partly undone the collapse onto the majority voice — what the per-target calibration of the style reward was meant to do — but has not replaced it with the requested voices: the other eight authors are unchanged or lower, at 0.000–0.062. The feature-level reward moves the surface statistics it measures towards the target's per-poem distribution, but those statistics do not by themselves make a poem identifiable as that author.

The judge agrees in direction, not in size. The held-out judge (§4.4) also scores GRPO below SFT, but by less: top-1 0.158 → 0.130, against 0.158 → 0.079 for the reward classifier. That v1's loss is larger on the classifier it was rewarded with than on an independent one is the opposite of gaming; but by neither measure did GRPO improve author identification. Per target, the judge's picture matches the classifier's:

Target author	SFT	GRPO v1	Gemma teacher
Блаже Конески	0.553	0.447	0.979
Пијан Славеј	0.465	0.442	0.698
Ефтим Клетников	0.383	0.298	0.872
Веле Смилевски	0.125	0.042	0.812
Влада Урошевиќ	0.021	0.064	0.681
Адем Гајтани	0.000	0.021	0.404
Ристо Лазаров	0.021	0.021	0.255
Славко Јаневски	0.020	0.000	0.240
Братислав Ташковски	0.022	0.000	0.109
Катица Ќулавкова	0.000	0.000	0.022

Judge top-1, per requested target author.

The judge has a majority-author prior of its own: it assigns 32.1 % of the teacher's outputs, and 36.5 % of SFT's, to Конески, and it identifies Ќулавкова almost never even in the teacher's output (0.022). Per-author judge figures for the smallest-profile targets are therefore a weak signal in either direction; the aggregate comparison between systems on the same pairs is the reliable one.

7.2 Partial copying

The copy gate held, but the policy moved to its edge. The share of outputs with at least one near-copied source line rose from 8.1 % to 44.4 %, although mean copy novelty is still 0.956 and only 3.4 % of outputs fall below 0.8. Part of the content gain comes from this: outputs containing a copied line have content-word recall 0.382, those without 0.302. The gate multiplies the content reward by the copied share, so one copied line in twenty costs only 5 % of R_{content}, and can be worth taking when it raises the line-level F1 by more. Reuse of source phrases below the gate's threshold is also visible on inspection ("не слетувај", "предели неминати" carried into an otherwise new poem). This is a modest form of reward hacking.

7.3 Training dynamics

The training reward (at the training weights) rose from 0.248 over the first 50 steps to 0.340 by step 200 and only to 0.361 by step 500. The content term kept improving slowly after step 200 (0.215 $\rightarrow$ 0.244) while copy novelty and the classifier stayed at their step-200 levels. The step-300 checkpoint confirms that later steps trade along one frontier: against step 500 it has lower R (0.381 vs. 0.406) and content recall (0.306 vs. 0.338), but copies less (38.7 % of outputs with a copied line vs. 44.4 %; 1.5 % below copy novelty 0.8 vs. 3.4 %), with the same author identification (top-1 0.088 vs. 0.079) and degeneracy (6.6 % vs. 5.3 %). The last 200 steps bought content recall partly by copying more.

7.4 The teacher on the same pairs

The teacher column of the v1 table is Gemma 4 31B on the val pairs themselves (`teacher_val.csv`), the upper reference for what the students imitate. Three observations.

GRPO v1 out-scores its teacher on the reward. $R = 0.406$ against 0.395. The teacher wins the style term by a wide margin (0.686 vs. 0.511) but scores only 0.105 on gated content, below SFT (0.123) and a third of GRPO's 0.302; its content-word recall (0.159) is below both students'. The teacher writes a new poem on the source's theme rather than carrying its lines over, and the line-level content reward scores that as barely above an unrelated poem. A higher R than the teacher is therefore not a sign that the student surpassed it; it shows that at $\alpha = \beta = 0.5$ the reward values content preservation the teacher never attempted.

The style gap is the open problem. On both classifiers the teacher is identified as the requested author three to eight times as often as either student (classifier top-1 0.641 vs. 0.079–0.158; judge 0.506 vs. 0.130–0.158). The judge reads the teacher's poems as the target at 76 % of the rate it reads real unseen poems (0.506 vs. 0.663). Author identity is what the teacher has and distillation did not transfer.

The teacher is not degenerate. 0.2 % degenerate outputs, line uniqueness 0.998: the looping in SFT output is the student's, not inherited from the data.

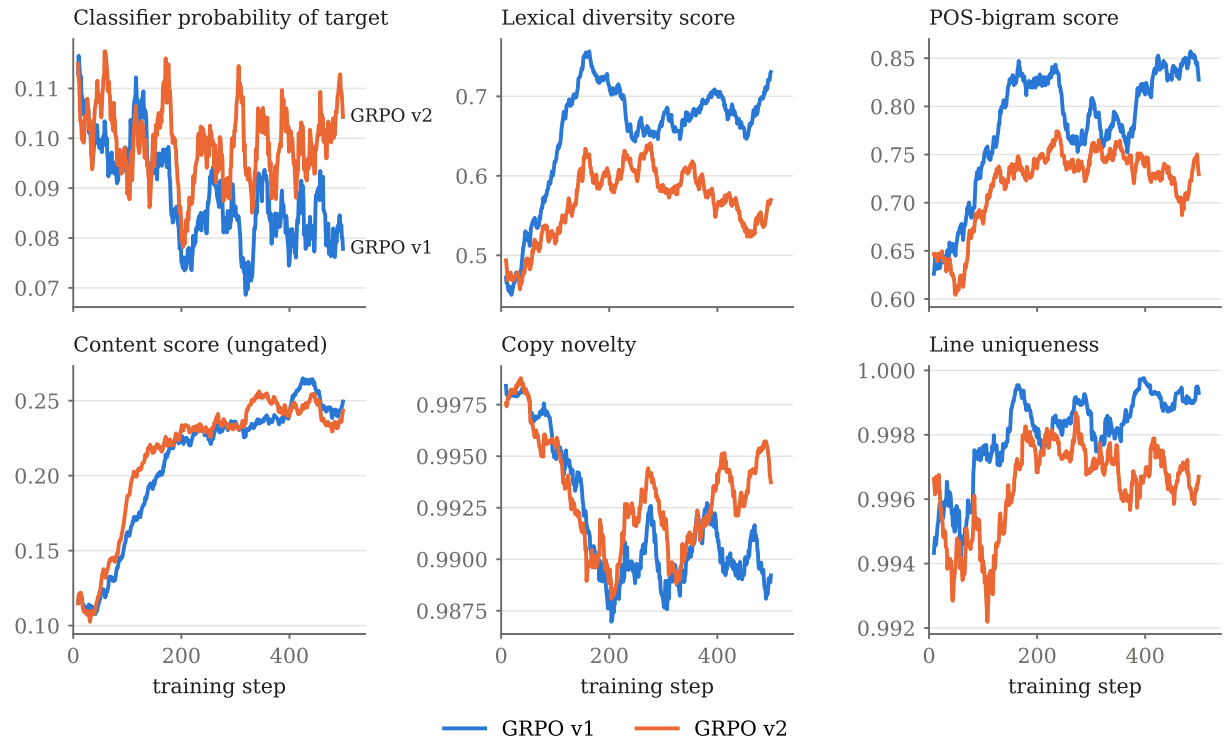


Figure 1: Reward components during GRPO training, per step (mean over the 32 scored completions, 25-step moving average), from each run's `reward_components.jsonl`. Only components on a common scale are shown: v2 reweights the classifier group and cubes the copy gate, so R, R_{style} and the gated content term are not comparable between runs. Produced by `scripts/plot_grpo_curves.py`.

8 GRPO v2: correcting the reward

v1's two failures are both properties of the reward, so v2 changes the reward and nothing else: same start policy, prompts, hyperparameters, seed and $\alpha/\beta = 0.3/0.7$ (`slurm/train_grpo_v2.slurm`, `models/qwen3-grpo-v2/`). A v1-vs-v2 difference can therefore be attributed to the two changes.

Classifier weight 5.6. The classifier group's within-group standard deviation is 0.051, against a mean of 0.288 for lexical diversity and POS bigrams. Weighting it by the ratio, $0.288/0.051 \approx 5.6$, gives it the same influence on the group-normalised advantage as either of those groups, instead of a fifth of it.

Copy gate exponent 3. The content reward is multiplied by $\texttt{copy_novelty}^3$ instead of `copy_novelty`. One copied line in twenty then costs $1 - 0.95^3 \approx 14\,\%$ of the content reward instead of 5 %, two lines 27 % instead of 10 %. A fully original poem is unaffected.

The risk v2 takes. Raising the classifier's weight makes it the largest single style term, so v2 is exactly the setting in which the reward classifier can be gamed; this is why the judge (§4.4) exists. v2 succeeds only if judge top-1 rises with classifier top-1.

8.1 Results

Metric (higher is better unless noted)	SFT (merged)	GRPO v1	GRPO v2	Real poems
R ($\alpha = \beta = 0.5$)	0.254	**0.406**	0.350	—
R^g_{style} / R^g_{content}	0.384 / 0.123	**0.511** / **0.302**	0.414 / 0.285	—
classifier top-1 = target	**0.158**	0.079	0.135	0.883†
judge top-1 = target	**0.158**	0.130	0.150	0.663†
share of outputs read as Конески (lower)	36.3%	**21.8%**	35.9%	—
outputs with ≥ 1 copied line (lower)	**8.1%**	44.4%	33.8%	—
copy novelty	**0.995**	0.956	0.966	—
content-word recall	0.174	**0.338**	0.270	—
degenerate outputs (lower)	30.1%	**5.3%**	25.4%	6.6%
structural fit	**0.846**	0.831	0.820	0.768

† held-out ceilings. Share read as Конески is the reward classifier's; the judge's is 36.5 %, 31.8 % and 39.1 %. Bold marks the best student per row.

v2 scores above its start policy on 81.8 % of the paired items, but above v1 on only 25.2 %: by the neutral reward it is the worse of the two GRPO runs. Per target author, on both classifiers:

	classifier top-1				judge top-1			
Target author	SFT	v1	v2	teacher	SFT	v1	v2	teacher
Блаже Конески	0.745	0.404	0.638	0.979	0.553	0.447	0.660	0.979
Пијан Славеј	0.372	0.186	0.442	0.651	0.465	0.442	0.419	0.698
Ефтим Клетников	0.149	0.021	0.043	0.660	0.383	0.298	0.277	0.872
Веле Смилевски	0.125	0.062	0.083	0.938	0.125	0.042	0.042	0.812
Влада Урошевиќ	0.021	0.021	0.000	0.787	0.021	0.064	0.043	0.681
Адем Гајтани	0.085	0.000	0.021	0.319	0.000	0.021	0.000	0.404
Ристо Лазаров	0.064	0.043	0.085	0.745	0.021	0.021	0.085	0.255
Славко Јаневски	0.000	0.020	0.020	0.160	0.020	0.000	0.000	0.240
Братислав Ташковски	0.022	0.022	0.022	0.609	0.022	0.000	0.000	0.109
Катица Ќулавкова	0.022	0.022	0.022	0.587	0.000	0.000	0.000	0.022

The classifier was not gamed. v2 is the setting in which gaming was most likely, and the judge rules it out: it moves with the reward classifier, by less (classifier top-1 $0.079 \rightarrow 0.135$, judge $0.130 \rightarrow 0.150$), and among the outputs the reward classifier attributes to the target, the judge agrees on 57 % for v2, against 62 % for SFT, 54 % for v1 and 64 % for the teacher. The character-n-gram reward is not being satisfied by surface tricks a word-level classifier would reject.

The difference from v1 is the majority voices. Almost all of v2's gain over v1 is on the two largest authors, Конески ($0.404 \rightarrow 0.638$) and Пијан Славеј ($0.186 \rightarrow 0.442$), and the share of all outputs the reward classifier assigns to Конески returns from 21.8 % to 35.9 %, the SFT level. The other eight targets stay at 0.000–0.085 on both classifiers, as they were under SFT and v1. Writing like the corpus's largest author is the cheapest way to keep the classifier term up, and weighting that term by 5.6 held the policy there. Neither GRPO run beats SFT on author identification by either measure.

The repetition fix was given back. Degenerate outputs rise from 5.3 % (v1) to 25.4 %, close to SFT's 30.1 %. v1's repetition fix came from the lexical-diversity and POS-bigram groups (§7); at classifier weight 5.6 those groups' share of the style reward falls from 2/6 to 2/10.6, and their pull on the policy with it.

The steeper copy gate partly worked. Outputs with a copied line fall from 44.4 % to 33.8 %, and those below copy novelty 0.8 from 3.4 % to 2.1 %, still well above SFT's 8.1 % and 0.4 %. Content-word recall falls with them ($0.338 \rightarrow 0.270$; 0.320 in outputs with a copied line, 0.244 without), confirming that part of v1's content gain was copying.

Why reweighting cannot fix the collapse. GRPO's update is driven by differences *within* a group of four samples for the same prompt. For eight of the ten targets, the student's samples are almost never read as the target author by either classifier, before or after GRPO, so all four score near zero on that term and the group-normalised advantage has nothing to amplify, however large its weight. The only targets on which the classifier term varies within a group are the ones the student already writes like — the two largest authors — and that is where both GRPO runs moved. The teacher shows the voices are reachable (judge top-1 0.506 against the 0.663 real-poem ceiling), but that capacity did not survive distillation into a 4B student from 1,146 examples. A student whose SFT distribution already puts some probability on the minority voices is the precondition for GRPO to help; §9 tests whether a different 4B model provides it.

The training log agrees. Over training (Figure 1, first versus last 50 steps), v2's classifier term stays flat, $0.106 \rightarrow 0.103$, where v1's fell, $0.102 \rightarrow 0.078$. The 5.6 weight did not move the policy *towards* the requested authors; it stopped it moving away from the voices it already had. Lexical diversity ($0.48 \rightarrow 0.56$, against v1's $0.49 \rightarrow 0.71$) and POS bigrams ($0.63 \rightarrow 0.72$, against $0.64 \rightarrow 0.84$) rose far less than in v1, which is the repetition fix given back. The ungated content term rose almost identically in both runs ($0.12 \rightarrow 0.24$ and $0.12 \rightarrow 0.25$); the cubed gate kept more of the trained completions copy-free (copy novelty 0.994 at the end, against 0.990).

9 A second student: Gemma 4 E4B

9.1 Purpose

Both of Qwen's open problems — collapse onto the largest authors and weak author identification after GRPO — could be properties of Qwen3-4B rather than of the data and reward. A second student of matched size from another family separates the two: if it collapses the same way, the cause is the method; if it does markedly better, the base model matters, and the size of the difference says by how much. Only the student changes — the same 1,146 synthetic examples, task format, hyperparameters, GRPO-v2 reward, 468 val pairs and seeds.

`google/gemma-4-E4B-it` (about 4.5B effective parameters, 8B with per-layer embeddings, instruction-tuned, Apache 2.0) is the closest match to Qwen3-4B. It is the teacher's own family, which may make the teacher's distribution easier to imitate; this is a stated confound, and also the realistic deployment choice.

9.2 Implementation

Everything family-specific lives in one registry, `model_families.py` (hub name, output-directory tag, LoRA targets, loss masking, loader, decoding overrides). The SFT, merge, generation, GRPO and reward-inspection scripts take `--family {qwen3,gemma4}`; the default resolves to exactly the pre-existing Qwen values, which tests pin, so v1 and v2 stay reproducible. Adapters and merged models record their

family, and loading one into the other family is refused. Facts that forced changes, verified against the real E4B configuration:

- *Loader.* `AutoModelForCausalLM` returns the multimodal class with vision and audio towers, so Gemma is loaded text-only (`Gemma4ForCausalLM` with the text config). The hub checkpoint stores the language weights under `model.language_model.*`, and transformers 5.9 has no conversion for it: a plain text-only `from_pretrained` *silently* leaves every language weight at its random initialisation. The loader maps the prefix, disables the library's weight-name conversion (which otherwise makes `save_pretrained` write keys that lose every weight on reload), and raises on any missing weight; a test exercises this on a tiny checkpoint of the real architecture.
- *LoRA targets.* The towers reuse the names `q_proj`…`down_proj` for wrapper modules PEFT cannot adapt, so the suffix list Qwen uses is safe only text-only; the multimodal fallback uses a regex restricted to the language model (258 modules over 42 layers — the last 18 share KV and have no `k_proj`/`v_proj`). The per-layer-embedding projections are never targets, for parity with Qwen's attention+MLP set.
- *Loss masking.* TRL ships no Gemma 4 training template, so `assistant_only_loss` is unavailable. Gemma trains on prompt–completion rows with `completion_only_loss`. Gemma's `eos_token` is `<eos>` but its turn ends with `<turn|>`, so the end token is set explicitly to `<turn|>`: otherwise SFT would train `<turn|><eos>`, and GRPO, which stops sampling on the tokenizer's end token, would sample past the end of the turn. Both formats then put the loss on the poem and its end-of-turn token; Qwen's additionally trains its constant empty reasoning block, a difference pinned by a test.
- *Decoding.* Gemma ships `top_k`=64; Gemma runs pin 20 to match Qwen.
- *Sequence length.* The smallest of 1024, 2048 and 4096 that truncates no training example under Gemma's tokenizer. Ten of the 1,146 examples exceed 2048 Gemma tokens, so Gemma trains at 4096 and truncates nothing, against Qwen's 6.2 % at 1024.

A compatibility job (`gemma_compat.py`) runs first: load, LoRA target count, template checks, training-length distribution, one generation, a 3-step SFT with a merge round-trip, and a GRPO smoke run.

The first run (`data/gemma_compat_report.json`) passed five of its six checks. The text-only loader loaded every language-model weight; the LoRA targets matched 258 modules (q/o/gate/up/down $\times$ 42, k/v $\times$ 24); the template's end-of-turn token was detected as `<turn|>`, and the untuned model's sample poem ended with it and contained no reasoning channel; and the length check measured a median of 537 Gemma tokens per training example, 31 over 1024, 10 over 2048 and none over 4096 (maximum 3,370), confirming 4096. The SFT–merge round-trip failed: on one training completion of 323 tokens, the merged model's log-probability was -1115.1 against the adapter model's -1071.3 (0.136 per token, against a tolerance of 0.02), and the GRPO smoke run, which follows it, did not run. **[pending: diagnosis of the merge round-trip (base vs. adapter vs. bf16 and fp32 merge, Qwen merge as control), then the GRPO smoke run]**

9.3 Results

Metric	Qwen3-4B base	Qwen3-4B SFT	Qwen3-4B GRPO-v2	Gemma 4 E4B base	Gemma 4 E4B SFT	Gemma 4 E4B GRPO-v2
line novelty	0.224	1.000	0.999	?	?	?
degenerate outputs	18.6%	30.1%	25.4%	?	?	?
classifier top-1	0.009	0.158	0.135	?	?	?
judge top-1	0.064	0.158	0.150	?	?	?
share read as Конески	8.5%	36.3%	35.9%	?	?	?
content-word recall	0.901[§]	0.174	0.270	?	?	?
R ($\alpha = \beta = 0.5$)	0.173	0.254	0.350	?	?	?

[pending: Gemma per-author table and interpretation: does it collapse onto Конески/Славеј like Qwen]

10 Limitations

- *Validation, not test.* Every number here is on the val split, which was also used to make decisions (decoding, v2's reward). The test split has not been touched; a final run of every system on it is the unbiased estimate. **[pending: test-split runs, or state explicitly that they are out of scope]**
- *Automatic metrics only.* Author identity is measured by two classifiers and fluency by proxies. No human (or native-speaker) rating of style, fluency or content has been collected.
- *Reward classifier and palettes saw val/test poems.* `classifier.pkl` and the TF-IDF palettes were fit on the whole corpus. They are author-level and never see the target of a transfer, but they are a leak path; the judge is the clean measurement.
- *No held-out SFT loss.* The SFT run monitored only training loss (§3.3), so the number of epochs was not chosen by validation, and the extent of memorisation in epochs 2–3 is unmeasured.
- *Qwen SFT truncation.* 6.2 % of Qwen's SFT examples were truncated at 1024 tokens and not retrained, to keep both GRPO runs' start policy fixed.
- *Single seed.* Each training run is one seed; differences of a few points between runs (e.g. classifier top-1 0.079 vs. 0.088 for v1's two checkpoints) are within run-to-run noise.
- *Content measurement is weak.* Unrelated poems already reach line-F1 0.553, and the teacher itself preserves little content-word recall (0.176), so the content signal is narrow.
- *Another Cyrillic language* (Serbian, Russian) would pass the validity gate and score well on multilingual content similarity; not observed in any output.
- *Same-family teacher* for the Gemma student (§9).

11 Reproducibility

Artifact	Produced by
splits	`src/make_splits.py` → `data/splits/poem_splits.csv`
synthetic train data	`scripts/generate_synthetic_parallel.sh` → `data/synthetic/synthetic_dataset.csv`
teacher on val	`scripts/generate_synthetic_parallel.sh 50 10 1 42 val` → `data/synthetic/teacher_val.csv`
SFT adapter	`slurm/train_sft.slurm` → `models/qwen3-lora-sft/`
base vs. SFT	`slurm/compare_base_lora.slurm`; decoding ablation `slurm/decoding_ablation.slurm`
reward calibration	`src/build_reward_calibration.py` → `data/reward_calibration.json` (deterministic)
reward checks	`src/evaluate_reward.py --fixtures`; `slurm/reward_groups.slurm` → `data/grpo/reward_groups.csv`
GRPO v1 / v2	`slurm/train_grpo.slurm`, `slurm/train_grpo_v2.slurm`; evaluation `slurm/eval_grpo.slurm`
Gemma student	`slurm/gemma_compat.slurm`, then the same jobs with `FAMILY=gemma4`, `slurm/train_grpo_gemma.slurm`
tables	`src/compare_generations.py --rewards --corpus-reference` over `data/validation_generations/*.csv`

Generation CSVs are tracked, because sampled model output cannot be regenerated bit-for-bit on different hardware; the scores can, and rerunning `compare_generations.py` on the tracked CSVs reproduces every table on CPU. The per-author breakdowns, the matched-budget vocabulary counts, the held-out cross-validated accuracy, the training-length statistics and the training-curve summaries were computed from the same files with one-off scripts.